



**Nombre**

Angel Alam

**Apellido**

Mateo encarnación

**Matrícula**

2024-0686

**Asignación**

Tarea 3

**Materia**

Programación para mecatronicos

**Profesor**

Carlos Antonio Pichardo Viuque

# Introducción

Al empezar a leer el capítulo 4 del libro, me di cuenta de algo en la carrera muchas veces nos lanzamos a escribir líneas de código sin detenernos a pensar en la lógica real que hay detrás. En la clase de Programación para Mecatrónicos siempre estamos lidiando con sensores, motores y condiciones, y justamente este capítulo sobre Lógica Matemática trata sobre cómo estructurar el pensamiento para que la máquina entienda exactamente qué queremos hacer sin enredarnos nosotros mismos.

En este ensayo quiero resumir las ideas principales de este capítulo (que trata sobre la lógica proposicional) y explicar cómo lo relaciono con lo que vemos en el día a día al programar circuitos y automatizar procesos, ya que al final todo se reduce a decisiones de verdadero y falso.

## 1. Las Proposiciones:

Lo primero que plantea el capítulo es el concepto de la proposición. Básicamente, una proposición es una oración que solo puede ser verdadera o falsa, sin puntos intermedios. No puede ser una pregunta ni una opinión. En el mundo de la programación y la electrónica, esto es el pan de cada día: o un pin del microcontrolador está en estado ALTO (True) o está en estado BAJO (False). O se presionó el botón o no se presionó.

En el libro te enseñan a representar estas frases con letras como p o q. Al principio parece álgebra de colegio, pero luego entiendes que es la misma base de las variables booleanas que usamos al programar.

## 2. Conectivos lógicos en el código

Una sola condición casi nunca resuelve un problema real, así que tenemos que combinarlas. El libro nos habla de los conectivos lógicos, que son literalmente los operadores que usamos en los lenguajes de programación. Tenemos la conjunción (que sería nuestro operador &&), la disyunción (que es el ||) y la negación (el símbolo !).

Por ejemplo, si quiero que un motor arranque, la lógica puede ser: "tiene que haber energía Y el botón de inicio debe estar presionado". Si entiendes bien cómo mezclar estos conectivos, te ahorras muchos dolores de cabeza cuando tienes que meter varios if o while en el código, evitando que el programa haga cosas raras.

## 3. Para qué sirven las tablas de verdad

Honestamente, hacer tablas de verdad a mano puede parecer tedioso al principio, pero el capítulo muestra que es la forma más segura de evaluar todas las posibilidades de un sistema para que no se te escape ningún error. Si tengo tres sensores en un circuito, la fórmula me dice que tengo 8 combinaciones posibles (2 al cubo). Hacer la tabla te ayuda a ver qué pasará con tu actuador en cada uno de esos 8 escenarios.

Además, me pareció muy interesante cuando el libro menciona conceptos como la tautología (cuando algo siempre es verdadero, como si hiciéramos un bucle infinito a propósito) y la contradicción (cuando creas una condición imposible sin darte cuenta y esa parte de tu código jamás llega a ejecutarse).

## **4. Equivalencias lógicas para no enredarse**

Esto de las equivalencias lógicas es súper útil para optimizar el código. A veces uno escribe unas condiciones larguísimas que, si te toca revisarlas una semana después, ni tú mismo las entiendes. Las equivalencias, como las famosas Leyes de De Morgan, te enseñan trucos matemáticos para simplificar esas expresiones.

Es decir, decir "no es cierto que el sensor A y el sensor B estén apagados" es lógicamente lo mismo que decir "el sensor A está encendido o el sensor B está encendido". Matemáticamente el resultado es igual, pero en el código una versión se lee de forma mucho más limpia y directa que la otra.

## **5. Reglas de inferencia y resolución de problemas**

En la última parte del capítulo se habla de cómo sacar conclusiones válidas a partir de ciertas premisas (argumentos). Tienen nombres súper formales como Modus Ponens o Modus Tollens, pero en la práctica es puro sentido común aplicado a la resolución de problemas técnicos.

Si yo sé que "si la temperatura sube a 50 grados, el ventilador debe encenderse", y luego noto que el ventilador no está encendido, mi cerebro (y el código) concluye lógicamente que la temperatura no ha llegado a 50 grados. En mecatrónica, esto es clave para diagnosticar fallas. Si algo no funciona, vas deduciendo lógicamente dónde está el error descartando cosas paso a paso.

## **Conclusión**

En resumen, leer este capítulo 4 me dejó claro que la matemática para la computación no son solo números, sino la forma en la que estructuramos las ideas. Entender bien la lógica proposicional hace la diferencia entre escribir código a lo loco, probando a ver si funciona de casualidad, y programar de manera pensada y analítica.

Todo lo que hacemos en Programación para Mecatrónicos, desde leer un simple sensor hasta controlar un sistema completo, pasa por ese filtro de verdaderos y falsos. Si se dominan estas reglas básicas de la lógica, programar se vuelve mucho más intuitivo y, definitivamente, se cometen menos errores en la lógica del sistema.